

```

/**
 * server/services/stuck-scheduler-recovery.ts (hardened)
 *
 * Fixes applied vs original:
 * - Singleton lock prevents overlapping scans.
 * - CommandCenter approval required before any recovery action.
 * - YouTube-only enforcement – skips non-youtube platform items.
 * - Skips demo/test/reviewer user IDs.
 * - Skips items when YouTube quota breaker is active.
 * - Skips items when channel needs_reconnect is true.
 * - Sets deferred status (not pending) when quota/token blocked.
 * - Log suppression – one summary per cycle, not one line per item.
 * - Proper escalated_at / recovered_at timestamps.
 * - No reset if item is already published/cancelled/permanent_fail.
 */

import { db } from '../db.js';
import { sql } from 'drizzle-orm';
import { createLogger } from '../lib/logger.js';
import { setJitteredInterval } from '../lib/timer-utils.js';
import { CommandCenter } from '../lib/command-center.js';
import { isProductionAutomationAllowed } from '../lib/production-guard.js';

const log = createLogger('stuck-scheduler-recovery');

// — Singleton lock —————

let isRunning = false;

// — Helpers —————
// Phantom/demo/test user detection delegated to centralized production guard

async function isQuotaBreakerActive(): Promise<boolean> {
  try {
    const result = await db.execute<{ tripped: boolean }>(sql`
      SELECT tripped FROM youtube_quota_state
      ORDER BY recorded_at DESC LIMIT 1
    `);
    return result.rows[0]?.tripped ?? false;
  } catch { return false; }
}

// Fix #5: full YouTube channel health check – replaces the weak needs_reconnect-only helper
// Validates: channel row exists, channel_id is real, tokens present, no reconnect flag

async function getYouTubeChannelHealth(userId: string): Promise<{
  ok: boolean;
  reason: string;

```

```

}> {
  try {
    const result = await db.execute<{
      channel_id:      string | null;
      access_token:    string | null;
      refresh_token:   string | null;
      needs_reconnect: boolean;
    }>(sql`
      SELECT channel_id, access_token, refresh_token, needs_reconnect
      FROM channels
      WHERE user_id = ${userId}
        AND platform = 'youtube'
      LIMIT 1
    `);

    const ch = result.rows[0];
    if (!ch) return { ok: false, reason: 'No YouTube channel row for this user' };

    if (!ch.channel_id) return { ok: false, reason: 'channel_id is null' };
    if (ch.channel_id.startsWith('UCdemo') ||
        ch.channel_id.toLowerCase().includes('demo') ||
        ch.channel_id.toLowerCase().includes('test') ||
        ch.channel_id === 'UC_test123')
      return { ok: false, reason: `Placeholder channel_id:
${ch.channel_id}` };
    if (ch.needs_reconnect) return { ok: false, reason: 'Channel needs_reconnect is true' };
    if (!ch.access_token) return { ok: false, reason: 'access_token is null' };
    if (!ch.refresh_token) return { ok: false, reason: 'refresh_token is null' };

    return { ok: true, reason: 'ok' };
  } catch (err: any) {
    return { ok: false, reason: `Channel health check failed: ${err.message?.slice(0, 80)}` };
  }
}

```

// — Core recovery —————

```

export async function recoverStuckScheduledItems(): Promise<void> {
  if (isRunning) {
    log.debug('[StuckSchedulerRecovery] Previous run still in progress – skipping');
    return;
  }

  // CommandCenter approval
  const approved = await CommandCenter.requestApproval('stuck_scheduler_recovery_scan', {
    description: 'Scan autopilot_queue for items stuck in scheduled status > 3 hours',
  }).catch(() => false);

  if (!approved) {
    log.debug('[StuckSchedulerRecovery] CommandCenter denied – skipping cycle');
  }
}

```

```

return;
}

isRunning = true;

try {
  const quotaBlocked = await isQuotaBreakerActive();
  let   recovered      = 0;
  let   deferred       = 0;
  let   escalated       = 0;
  let   skipped        = 0;

  const stuckItems = await db.execute<{
    id:          number;
    type:        string;
    user_id:     string;
    platform:    string;
    status:      string;
    scheduled_at: Date;
    miss_count:  number;
    overdue_ms:  number;
  }>(sql`
    SELECT
      id,
      type,
      user_id,
      COALESCE(platform, target_platform, 'unknown') AS platform,
      status,
      scheduled_at,
      COALESCE(miss_count, 0) AS miss_count,
      EXTRACT(EPOCH FROM (NOW() - scheduled_at)) * 1000 AS overdue_ms
    FROM autopilot_queue
    WHERE status = 'scheduled'
      AND scheduled_at < NOW() - INTERVAL '3 hours'
    ORDER BY scheduled_at ASC
    LIMIT 50
  `);

  for (const item of stuckItems.rows) {
    const newMissCount = (item.miss_count ?? 0) + 1;
    const overdueMins = Math.round(item.overdue_ms / 60_000);

    // Item 3: strict YouTube-only platform enforcement
    // Allowed: 'youtube', 'unknown', null/empty (treated as YouTube)
    // Known non-YouTube: permanent_fail immediately
    // Any other named platform: permanent_fail (not a fall-through to YouTube)

    const platformNorm = (item.platform ?? '').toLowerCase().trim();

    const knownNonYoutube = ['tiktok', 'rumble', 'twitch', 'kick'].some(

```

```

p => platformNorm.startsWith(p)
);

if (knownNonYoutube) {
  await db.execute(sql`
    UPDATE autopilot_queue
    SET status          = 'permanent_fail',
        error_message = ${'Non-YouTube platform (' + item.platform + ') in YouTube-only
mode'},
        updated_at     = NOW()
    WHERE id = ${item.id}
  `);
  skipped++;
  continue;
}

// Any other named platform that isn't 'youtube' or 'unknown' → permanent_fail
// Do NOT silently promote to YouTube just because the user has a YouTube channel.
if (platformNorm && platformNorm !== 'youtube' && platformNorm !== 'unknown') {
  await db.execute(sql`
    UPDATE autopilot_queue
    SET status          = 'permanent_fail',
        error_message = ${'Unrecognised platform (' + item.platform + ') in YouTube-only
mode – permanent_fail'},
        updated_at     = NOW()
    WHERE id = ${item.id}
  `);
  skipped++;
  continue;
}

// Fix #8: centralized production guard replaces local phantom set
const prodGuard = isProductionAutomationAllowed(item.user_id, undefined, item.platform);
if (!prodGuard.allowed) {
  await db.execute(sql`
    UPDATE autopilot_queue
    SET status          = 'permanent_fail',
        error_message = ${'Blocked by production guard: ' + prodGuard.reason},
        updated_at     = NOW()
    WHERE id = ${item.id} AND status = 'scheduled'
  `);
  skipped++;
  continue;
}

// Fix #4: escalate at miss >= 5 (was > 5, which escalated at miss 6)
if (newMissCount >= 5) {
  await db.execute(sql`
    UPDATE autopilot_queue
    SET status          = 'escalated',

```

```

        miss_count      = ${newMissCount},
        escalated_at    = NOW(),
        error_message = ${'Escalated after ' + newMissCount + ' misses (' + overdueMins +
' min overdue)'},
        updated_at      = NOW()
    WHERE id = ${item.id} AND status = 'scheduled'
`);
    escalated++;
    continue;
}

// Fix #5: full channel health – only reset to pending when channel is truly ready
const channelHealth = await getYouTubeChannelHealth(item.user_id);
if (quotaBlocked || !channelHealth.ok) {
    const reason = quotaBlocked
        ? 'YouTube quota breaker active'
        : channelHealth.reason;

    await db.execute(sql`
        UPDATE autopilot_queue
        SET status          = 'deferred',
            miss_count      = ${newMissCount},
            deferred_until = NOW() + INTERVAL '2 hours',
            error_message = ${reason + ' – deferred by stuck-scheduler (miss #' + newMissCount
+ ')}',
            updated_at      = NOW()
        WHERE id = ${item.id} AND status = 'scheduled'
    `);
    deferred++;
    continue;
}

// CommandCenter approval per-action
const actionApproved = await CommandCenter.requestApproval(
    'reset_stuck_item_to_pending',
    { itemId: item.id, type: item.type, overdueMins }
).catch(() => false);

if (!actionApproved) {
    skipped++;
    continue;
}

await db.execute(sql`
    UPDATE autopilot_queue
    SET status          = 'pending',
        scheduled_at    = NOW(),
        miss_count      = ${newMissCount},
        recovered_at    = NOW(),
        error_message = ${'Recovered by stuck-scheduler: ' + overdueMins + ' min overdue

```

```

(miss #' + newMissCount+ ' ')}',
    updated_at    = NOW()
    WHERE id = ${item.id} AND status = 'scheduled'
`);
recovered++;
}

// Single summary log – not per-item
if (recovered + deferred + escalated + skipped > 0) {
  log.info(
    `[StuckSchedulerRecovery] Cycle complete – ` +
    `recovered: ${recovered}, deferred: ${deferred}, ` +
    `escalated: ${escalated}, skipped: ${skipped}`
  );
}

} finally {
  isRunning = false;
}
}

// — Lifecycle —————

let stopInterval: (() => void) | null = null;

export function startStuckSchedulerRecovery(): void {
  if (stopInterval) return;
  log.info('[StuckSchedulerRecovery] Starting – interval: 15 min');
  stopInterval = setJitteredInterval(
    () => recoverStuckScheduledItems().catch(err =>
      log.error('[StuckSchedulerRecovery] Cycle error:', err)
    ),
    15 * 60 * 1000,
  );
}

export function stopStuckSchedulerRecovery(): void {
  if (stopInterval) { stopInterval(); stopInterval = null; }
}

```